Meeting 5 — Software Component Architecture

Digital Harmonium Workstation Firmware

Phase 3.5 — System Module Design (Version 1.0)

Objective

Unlike the Runtime State document, which defines **what the system is doing**, this document defines **which permanent software modules exist inside the firmware**.

Each module owns one responsibility.

No module should perform multiple unrelated jobs.

The firmware should therefore resemble a collection of small specialists that cooperate rather than one giant program.

This modular architecture is the foundation for maintainability, debugging, testing, and future expansion.

Layer 1 — Hardware Interface Modules

These modules communicate directly with physical hardware.

They never make musical decisions.

Their responsibility is only to interact with devices.

---

1. Keyboard Scanner

Responsibility

Continuously scans the keyboard matrix and determines which keys are currently pressed or released.

Inputs

- Keyboard matrix rows
- Keyboard matrix columns

Outputs

- Key Press events
- Key Release events

Notes

This module knows nothing about instruments, samples, or sound generation.

---

2. Button Manager

Responsibility

Reads all dedicated control buttons.

Examples:

- Voice
- Tabla
- Tanpura
- Record
- Playback
- Menu
- Start/Stop

Outputs

High-level button events delivered to the UI Manager.

---

3. LED Manager

Responsibility

Controls every keyboard LED.

Responsibilities include

- Key illumination
- Playback guidance
- Recording indication
- Status indicators
- Startup LED test

---

4. Display Manager

Responsibility

Controls the 16×2 LCD.

Responsibilities

- Screen rendering
- Cursor positioning
- Menu updates
- Status messages

The rest of the firmware never writes directly to the LCD.

---

5. Audio Driver

Responsibility

Transfers audio buffers to the MAX98357 DAC through I^2S.

Responsibilities

- DMA management
- Audio timing
- Buffer switching

This is the lowest audio layer.

---

6. SD Card Manager

Responsibility

Provides file access.

Responsibilities

- Open files
- Read files
- Save recordings
- Browse instruments

It never interprets VHL data.

---

7. Flash Memory Manager

Responsibility

Loads active VHL instruments into internal memory.

Responsibilities

- Allocate RAM
- Copy instrument
- Release previous instrument

---

8. Power Manager

Responsibility

Supervises hardware startup and shutdown.

Responsibilities

- Initialization
- Safe reset
- Self-test
- Future battery support

---

9. USB Interface

Responsibility

Handles communication with a computer.

Current purpose

- Firmware upload
- Debugging

Future purpose

- File transfer
- MIDI
- USB maintenance

---

10. System Timer

Responsibility

Provides accurate timing for the entire firmware.

Used by

- Audio engine
- Recorder
- Metronome
- UI timing
- LED blinking

Layer 2 — Runtime Control Modules

These modules coordinate the musical behavior of the workstation.

---

11. System Manager

Responsibility

Master controller of the firmware.

Coordinates all major modules.

Responsible for

- Boot sequence
- Shutdown
- State transitions
- Overall supervision

---

12. User Interface Manager

Responsibility

Implements every screen and menu.

Uses

- Button Manager
- Display Manager

Responsible for

- Navigation
- Menu logic
- User interaction

---

13. Instrument Manager

Responsibility

Keeps track of the currently selected instrument.

Responsible for

- Voice selection
- Instrument switching
- Active VHL assignment

---

14. Voice Manager

Responsibility

Allocates voices whenever keys are pressed.

Responsible for

- Polyphony
- Voice stealing
- Voice recycling
- Active voice list

This is one of the most important modules.

---

15. Audio Engine

Responsibility

Produces the final audio stream.

Responsibilities

- Voice mixing
- Sample playback
- Envelopes
- Filters
- LFO
- DSP routing

Consumes active voices from the Voice Manager and generates the audio buffer sent to the Audio Driver.

---

16. Tabla Engine

Responsibility

Manages rhythmic accompaniment.

Responsible for

- Current taal
- Tempo
- Bol sequencing
- Start/Stop
- Synchronization

Uses recorded tabla samples.

---

17. Tanpura Engine

Responsibility

Generates the drone accompaniment.

Responsible for

- Drone pattern
- Tempo
- Transpose
- Start/Stop

Uses recorded tanpura samples.

---

18. Recorder Manager

Responsibility

Handles recording and playback of performances.

Responsible for

- Recording events
- Saving performances
- Playback
- LED-guided replay
- Timing synchronization

Only performance events are stored; audio is regenerated during playback.

---

19. Metronome Engine

Responsibility

Provides timing assistance independent of tabla.

Responsible for

- Tempo
- Beat generation
- Click playback
- Visual beat indication

Useful for practice sessions without rhythmic accompaniment.

Overall Software Architecture

```
Application Layer
│
├── System Manager
├── UI Manager
├── Instrument Manager
├── Voice Manager
├── Audio Engine
├── Recorder Manager
├── Tabla Engine
├── Tanpura Engine
```

```
└── Metronome Engine

_____________________________

Hardware Layer
│
├── Keyboard Scanner
├── Button Manager
├── LED Manager
├── Display Manager
├── Audio Driver
├── SD Manager
├── Flash Manager
├── USB Interface
├── Power Manager
└── System Timer
```

Design Principles

Every module must:

- Have one clearly defined responsibility.
- Expose a clean interface.
- Avoid direct knowledge of unrelated modules.
- Communicate through the System Manager or well-defined interfaces.
- Be independently testable.
- Be replaceable without affecting the rest of the firmware.

Summary

Meeting 5 establishes the permanent software architecture of the Digital Harmonium Workstation.

The firmware is divided into **19 core modules**, grouped into two logical layers:

- **10 Hardware Interface Modules**, responsible for interacting with physical devices.
- **9 Runtime Control Modules**, responsible for implementing musical behavior, user interaction, and audio generation.

This modular structure will serve as the blueprint for the Phase 3 firmware implementation and ensures that the system remains organized, scalable, and maintainable as additional instruments and features are introduced.